RPC Semantics in the Presence of Failure

Dr. Saurabh Bilgaiyan
Assistant Professor
School of Computer Engineering
KIIT Deemed to be University
Bhubaneswar

UNIT-2-Part: RPC Semantics in the Presence of Failure

Five different classes of failure that can occur in RPC systems:

- The client is unable to locate the server.
- The request message from the client to the server is lost.
- The reply message from the server to the client is lost.
- The server crashes after receiving a request.
- The client crashes after sending a request.

Client cannot locate the server

- The server might be down, for example. Alternatively, suppose that the client is compiled using a particular version of the client stub, and the binary is not used for a considerable period of time. In the meantime, the server evolves and a new version of the interface is installed and new stubs are generated and put into use. When the client is finally run, the binder will be unable to match it up with a server and will report failure.
- The problem remains of how this failure should be dealt with.
- Wth the server of Fig. 2-9(a), each of the procedures returns a value, with the code –1 conventionally used to indicate failure. For such procedures, just returning –1 will clearly tell the caller that something is amiss.
- In UNIX, a global variable, errno, is also assigned a value indicating the error type. In such a system, adding a new error type "Cannot locate server" is simple.
- The trouble is, this solution is not general enough. Consider the sum procedure of Fig. 2-19. Here –1 is a perfectly legal value to be returned, for example, the result of adding 7 to –8. Another error-reporting mechanism is needed.
- One possible candidate is to have the error raise an exception. In some languages (e.g., Ada), programmers can write special procedures that are invoked upon specific errors, such as division by zero.
- This approach, too, has drawbacks. To start with, not every language has exceptions or signals.

Lost Reply Messages

- Lost replies are considerably more difficult to deal with. The obvious solution is just to rely on the timer again. If no reply is forthcoming within a reasonable period, just send the request once more.
- The trouble with this solution is that the client's kernel is not really sure why there was no answer. Did the request or reply get lost, or is the server merely slow? It may make a difference.
- In particular, some operations can safely be repeated as often as necessary with no damage being done. A request such as asking for the first 1024 bytes of a file has no side effects and can be executed as often as necessary without any harm being done. A request that has this property is said to be idempotent.
- Now consider a request to a banking server asking to transfer a million dollars from one account to another. If the request arrives and is carried out, but the reply is lost, the client will not know this and will retransmit the message. The bank server will interpret this request as a new one, and will carry it out too. Two million dollars will be transferred. Heaven forbid that the reply is lost 10 times. Transferring money is not idempotent.
- One way of solving this problem is to try to structure all requests in an idem-potent way. In practice, however, many requests (e.g., transferring money) are inherently nonidempotent, so something else is needed.
- Another method is to have the client's kernel assign each request a sequence number. By having each server's kernel keep track of the most recently received sequence number from each client's kernel that is using it, the server's kernel can tell the difference between an original request and a retransmission and can refuse to carry out any request a second time.
- An additional safeguard is to have a bit in the message header that is used to distinguish initial requests from retransmissions (the idea being that it is always safe to perform an original request; retransmissions may require more care).

Server Crashes

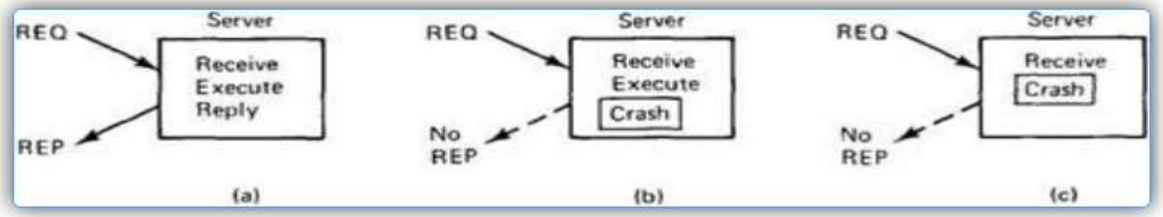

Fig. 2-24. (a) Normal case. (b) Crash after execution. (c) Crash before execution.

- The next failure on the list is a server crash. It too relates to idempotency, but unfortunately it cannot be solved using sequence numbers.
- Fig. 2-24(a)- A request arrives, is carried out, and a reply is sent. Now consider Fig. 2-24(b). A request arrives and is carried out, just as before, but the server crashes before it can send the reply. Finally, look at Fig. 2-24(c). Again a request arrives, but this time the server crashes before it can even be carried out.
- The annoying part of Fig. 2-24 is that the correct treatment differs for (b) and (c). In (b) the system has to report failure back to the client (e.g., raise an exception), whereas in (c) it can just retransmit the request. The problem is that the client's kernel cannot tell which is which. All it knows is that its timer has expired.
- Three schools of thought exist on what to do here. One philosophy is to wait until the server reboots (or rebinds to a new server) and try the operation again. The idea is to keep trying until a reply has been received, then give it to the client. This technique is called at least once semantics and guarantees that the RPC has been carried out at least one time, but possibly more.
- The second philosophy gives up immediately and reports back failure. This way is called at most once semantics and guarantees that the rpc has been carried out at most one time, but possibly none at all.
- The third philosophy is to guarantee nothing. When a server crashes, the client gets no help and no promises. The RPC may have been carried out anywhere from 0 to a large number of times. The main virtue of this scheme is that it is easy to implement.

Client Crashes

- What happens if a client sends a request to a server to do some work and crashes before the server replies? At this point a computation is active and no parent is waiting for the result. Such an unwanted computation is called an orphan.
- Orphans can cause a variety of problems. As a bare minimum, they waste CPU cycles. They can also lock files or otherwise tie up valuable resources. Finally, if the client reboots and does the RPC again, but the reply from the orphan comes back immediately afterward, confusion can result.
- What can be done about orphans? In **solution 1**, before a client stub sends an RPC message, it makes a log entry telling what it is about to do. The log is kept on disk or some other medium that survives crashes. After a reboot, the log is checked and the orphan is explicitly killed off. This solution is called extermination.
- The disadvantage of this scheme is the horrendous expense of writing a disk record for every RPC. Furthermore, it may not even work, since orphans themselves may do RPCs, thus creating grandorphans or further descendants that are impossible to locate. Finally, the network may be partitioned, due to a failed gateway, making it impossible to kill them, even if they can be located.
- In **solution 2**, called reincarnation, all these problems can be solved without the need to write disk records. The way it works is to divide time up into sequentially numbered epochs. When a client reboots, it broadcasts a message to all machines declaring the start of a new epoch. When such a broadcast comes in, all remote computations are killed. Of course, if the network is partitioned, some orphans may survive. However, when they report back, their replies will contain an obsolete epoch number, making them easy to detect.
- **Solution 3** is a variant on this idea, but less Draconian. It is called gentle reincarnation. When an epoch broadcast comes in, each machine checks to see if it has any remote computations, and if so, tries to locate their owner. Only if the owner cannot be found is the computation killed.
- **Solution 4**, expiration, in which each RPC is given a standard amount of time, T, to do the job. If it cannot finish, it must explicitly ask for another quantum, which is a nuisance. On the other hand, if after a crash the server waits a time T before rebooting, all orphans are sure to be gone. The problem to be solved here is choosing a reasonable value of T in the face of RPCs with wildly differing requirements.
- In practice, none of these methods are desirable. Worse yet, killing an orphan may have unforeseen consequences. For example, suppose that an orphan has obtained locks on one or more files or data base records. If the orphan is suddenly killed, these locks may remain forever. Also, an orphan may have already made entries in various remote queues to start up other processes at some future time, so even killing the orphan may not remove all traces of it.

Lost Request Messages

- The second item on the list is dealing with lost request messages.
- This is the easiest one to deal with: just have the kernel start a timer when sending the request. If the timer expires before a reply or acknowledgement comes back, the kernel sends the message again.
- If the message was truly lost, the server will not be able to tell the difference between the retransmission and the original, and everything will work fine.
- Of course, so many request messages are lost that the kernel gives up and falsely concludes that the server is down, in which case we are back to "Cannot locate server."

Q&A

Homework Questions

1) Selective short question 1
2) Selective short question 2
3) Selective long question1
4) **********
5) **********
6) **********
7) **********

Thank You!